2 parcial

Universidad: Politécnica de Atlautla

Programa: Ing. en Tecnologías de la Información e Innovación Digital

Asignatura: Fundamentos de Inteligencia Artificial

Tema: Actividad 2.1 Problemas de búsqueda, 2.2 Espacios de búsqueda.

Docente: Mtro. Javier García Lara

Alumno: Anahi Fuentes Avendaño

Enlace de GitHub:
https://github.com/fana60203/FIA_G171_Anahi_Fuentes

Grupo: 171

Fecha de entrega:

18 – Octubre - 2026

Tema: Actividad 2.1 Problemas de búsqueda, 2.2 Espacios de búsqueda.

Producto y Descripción: Modelar 2 problemas reales como problemas de búsqueda (estado, acciones, coste, meta). Representar cada espacio como grafo(dirigido/ponderado). Programar en Python un generador de estados y un grafo con NetworkX. Entregar diagrama del grafo, código comentado y análisis de complejidad.

Problema 1- Ruta de menor tiempo en el Metro de la CDMX

1. **Datos del problema**

Una estudiante sale de la estación Universidad (Línea 3) y quiere llegar a la estación Zócalo/Tenochtitlan (Línea 2) en el menor tiempo posible. Se modelan las líneas 1, 2 y 3 completas (65 combinaciones estación-línea). Datos de entrada:

Dato	Valor
Líneas modeladas	L1 (20 estaciones), L2 (24 estaciones), L3 (21 estaciones)
Tiempo entre estaciones consecutivas	2 min (valor medio aproximado, supuesto del modelo)
Tiempo de transbordo	5 min (caminar + espera, supuesto del modelo)
Correspondencias	Hidalgo (L2–L3), Balderas (L1–L3), Pino Suárez (L1–L2)
Origen/Destino	Universidad (L3) / Zócalo (L2)

2. **Planteamiento**

Existen al menos dos rutas razonables: (a) la Línea 3 hasta Hidalgo y transbordo a la Línea 2; (b) la Línea 3 hasta Balderas, transbordo a la Línea 1 hasta Pino Suárez y segundo transbordo a la Línea 2. La pregunta es cuál minimiza el tiempo total, y qué tan costoso es para cada algoritmo de búsqueda encontrarla. Como el costo depende del tipo de acción (avanzar o transbordar), el estado debe incluir la línea en la que viaja el usuario, no solo la estación.

3. Formulación

Componente	Definición
Estados S	s = (estación, línea). |S| = 65
Estado inicial	(Universidad, L3)
Acciones A(s)	R_A_, IR_A_ (para avanzar) en la misma línea TRANSBORDO_L (para cambiar de linea) si la estación pertenece a otra línea
Modelo de transición	IR_A: (e, l) → (e', l); TRANSBORDO_Ln: (e, l) → (e, n)
Costo de paso c (s, a, s')	2 min si IR_A; 5 min si TRANSBORDO (todos > 0)
Prueba de meta	estación == Zócalo (en cualquier línea)

4. Procedimiento

(1) Se codificó el problema como la clase RutaMetro con los cinco componentes.

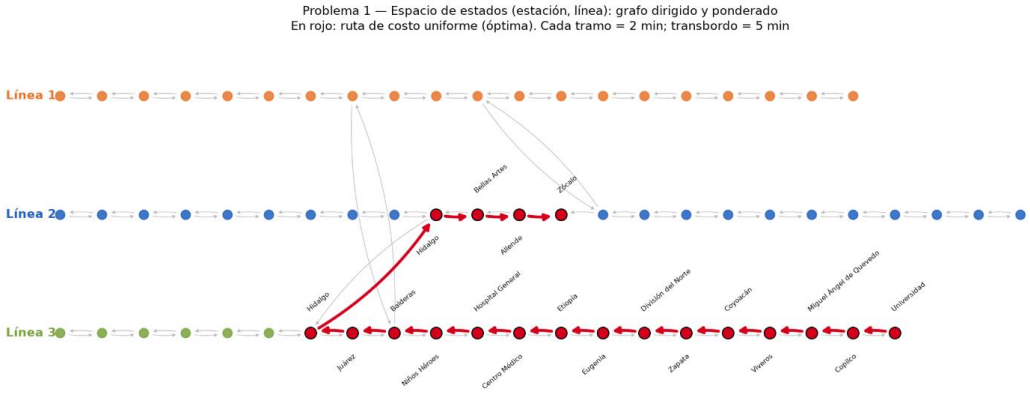
(2) Generar_espacio_estados exploró todos los estados y construyó un nx.DiGraph con peso en cada arista.

(3) Se ejecutaron BFS, DFS y UCS registrando costo, pasos, nodos expandidos y generados.

(4) Se verificó el costo de UCS con Dijkstra de NetworkX. (5) Se dibujó el grafo resaltando la ruta óptima.

5.Solucion

Ruta óptima (UCS): Universidad (L3) → 14 tramos en L3 hasta Hidalgo (28 min) → transbordo a L2 (5 min) → Bellas Artes → Allende → Zócalo (6 min). Costo total = 39 min, 18 acciones. El grafo tiene 65 estados y 128 transiciones dirigidas; el factor de ramificación promedio es b = 2.00.



Algoritmo	Costo (min)	Pasos	Expandidos	Generados	¿Optimo en costo?
BFS	39	18	30	36	Si, en esta instancia
DFS	39	18	33	35	Si, en esta instancia
UCS	39	18	35	41	Si garantizado

La alternativa por Balderas–Pino Suarez cuesta 12×2 + 5 + 3×2 + 5 + 2 = 42 min frente a 39 min de la ruta por Hidalgo, de modo que el segundo transbordo la penaliza. En esta instancia BFS y DFS coinciden con el óptimo, pero solo UCS lo garantiza: BFS minimiza el número de pasos (ambas rutas tienen 18) y DFS no ofrece garantía alguna.

6. Análisis de complejidad

Algoritmo	Tiempo	Espacio	Completo	Optimo
BFS	O(bd)	O(bd)	Si	Solo con costos iguales
DFS	O(bm)	O(b · m)	Si con lista de explorados	No

			(espacio finito)	
UCS	O(b1+⌊C*/ε⌋)	O(b1+⌊C*/ε⌋)	Si (ε = 2 > 0)	Si

Aquí b ≈ 2.00, la profundidad de la solución es d = 18, C* = 39 min y ε = 2 min (costo mínimo de paso), así que ⌊C*/ε⌋ = 19 es el cociente redondeado hacia abajo y 20 es el exponente final de la cota de UCS. Con búsqueda en grafo (lista de explorados) el trabajo queda acotado por el tamaño del espacio: BFS es O(|S| + |A|) = O (65 + 128) y UCS con montículo binario es O ((|S| + |A|) log |S|). Empíricamente, UCS expandió 35 de 65 estados (54%): al ser un espacio pequeño (|S| = 65) se explora casi completo, pero con la red completa del Metro (≈195 estaciones, 12 líneas) la cota bd sin lista de explorados sería inviable, y la lista de explorados es lo que mantiene el problema tratable.

Problema 2 – Robot de almacén: recoger y entregar un paquete.

1.Datos del problema

Un robot móvil autónomo (AMR) en un almacen de 5 × 7 celdas deben ir desde su base **R** hasta el anaquel **P**, recoger un paquete y entregarlo en el muelle **D**. Los estantes (#) son infranqueables y hay un pasillo congestionado (~) donde avanzar tarda el triple.

Dato	**Valor**
Cuadricula	5 filas × 7 columnas; movimientos N, S, E, O
Posiciones	R = (0,0), P = (4,1), D = (1,6) (fila, columna)
Costos	Mover a celda libre = 1 s; a celda ~ = 3 s; RECOGER = 2 s; ENTREGAR = 2 s

2. Planteamiento

Determinar la secuencia de acciones de menor tiempo total que lleve el paquete de P a D. El problema no se reduce a ir de R a D: el robot debe pasar por P *antes*

de entregar. Por eso el estado incluye una fase (sin paquete / con paquete / entregado), y las acciones RECOGER y ENTREGAR son irreversibles, lo que vuelve el grafo del espacio de estados genuinamente dirigido.

3. Formulación

Componente	Definición
Estado S	s = (fila, col, fase), fase ∈ {0: sin paquete, 1: con paquete, 2: entregado}. |S| = 59
Estado inicial	(0, 0, 0)
Acciones A(s)	N, S, E, O si la celda destino existe y no es #; RECOGER si (fila, col) =P y fase=0; ENTREGAR si (fila,col)=D y fase=1
Modelo de transición	Mover: (r, c, f) → (r+dr, c+dc, f); RECOGER: (P,0) → (P,1); ENTREGAR: (D,1) → (D,2)
Costo de paso	1 / 3 (celda ~) / 2 (RECOGER, ENTREGAR); todos > 0
Prueba de meta	fase == 2

4. Procedimientos

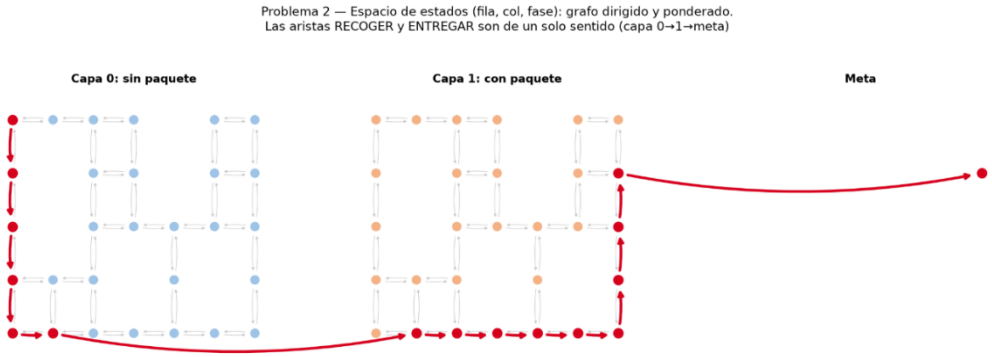
Igual que en el Problema 1: clase RobotAlmacen con los cinco componentes → generación exhaustiva del espacio de estados como nx.DiGraph ponderado → BFS, DFS y UCS con métricas → verificación con Dijkstra → diagramas. Los estados meta no se expanden, y como el costo depende de la celda destino, se evalúa en la arista (s, a, s').

5. Solución

Plan óptimo (UCS), costo = 17 s en 15 acciones: S S S S E RECOGER E E E E E N N N ENTREGAR. El robot baja por la columna 0 hasta la fila 4, recoge en P,

recorre el pasillo inferior hacia el este evitando la zona congestionada (que le habría costado 3 s por celda) y sube por la columna 6 hasta D.





Algoritmo	Costo (s)	Pasos	Expandidos	Generados	¿Optimo en costo?
BFS	21	15	57	59	No
DFS	29	23	50	50	No
UCS	17	15	57	59	Si garantizado

Aquí los algoritmos si difieren: BFS encuentra un plan de 15 pasos (el mínimo en número de acciones) pero de 21 s, porque atraviesa la zona congestionada; DFS devuelve un plan largo de 23 pasos y 29 s; únicamente UCS obtiene los 17 s óptimos. Esto muestra por que el costo debe formar parte de la definición del problema: minimizar pasos no es lo mismo que minimizar tiempo.

6. Análisis de complejidad

Para una cuadrícula de n × m celdas libres el espacio de estados tiene $|S| \leq 2 \cdot n \cdot m + 1$ estados (dos capas más la meta); aquí 2·(29 celdas libres) + 1 = 59. Cada estado tiene a lo sumo 4 movimientos más una acción especial, así que $b \leq 5$; el valor medio medido es $b = 2.59$ (con 150 aristas). El costo mínimo de paso es $\varepsilon = 1$ y $C^* = 17$, por lo que UCS es $O(b^{1+\lfloor C^*/\varepsilon \rfloor}) = O(b^{18})$ en el peor caso sin lista de explorados, mientras que con explorados el trabajo es $O((|S| + |A|) \log |S|) = O((59 + 150) \log 59)$, lineal-logarítmico en el tamaño del mapa. BFS y DFS usan $O(|S| + |A|)$ tiempo; el espacio es $O(|S|)$ para BFS/UCS y $O(|S|)$ para DFS con explorados. Empíricamente UCS expandió 57 de 59 estados.

Conclusiones

Lo más difícil fue conseguir un código que hiciera justo lo que pedía la actividad (modelar el problema, generar el espacio de estados y resolverlo) y, sobre todo, entenderlo bien y que se pudiera ver de forma fácil. Para eso me apoyé en Claude, y después revisé que el código corriera y que los resultados tuvieran sentido. También tuve que decidir qué lleva un estado: en el Metro no bastaba con la estación, había que saber en qué línea ibas; y en el robot importaba si ya traía el paquete o no. Ver el espacio de estados dibujado como grafo me ayudó mucho a entender cada problema. En el Metro se ven las líneas unidas por los transbordos, y en el almacén se ven las capas conectadas por flechas que no tienen vuelta (recoger y entregar).

Uso de IA

Herramienta Claude (Anthropic), modelo Claude Sonnet 5.5, interfaz de chat con ejecución de código.

Prompt: "Realiza lo que se indica en la actividad 2.1 Problemas de búsqueda y 2.2 Espacios de búsqueda con lo que nos indica en el apartado Producto y Descripción e Instrucciones Generales, siguiendo rigurosamente la lista de cotejo" (se adjuntaron imágenes de la actividad y de la lista de cotejo).

Uso: Generación del código Python (módulo de búsqueda, dos problemas, generación de grafos con NetworkX), de los diagramas y del borrador del texto de este reporte.

Referencias

NetworkX Developers. (s. f.). Traversal. NetworkX documentation. documentation.https://networkx.org/documentation/stable/reference/algorithms/shortest_paths/dijkstra.html?utm_source=.com

Patel, A. J. (2014). Introduction to the A* algorithm. Red Blob Games. Recuperado el 18 de octubre de 2026, de https://www.redblobgames.com/pathfinding/a-star/introduction.html

Código del Problema 1 comentado

A.1 problema1/problema1.py (Problema 1: marco de búsqueda + Metro CDMX + complejidad)

```
"""
problema1.py -- Actividad 2 (2.1 Problemas de búsqueda + 2.2 Espacios de búsqueda)
PROBLEMA 1: ruta de menor tiempo en el Metro de la CDMX (Universidad -> Zócalo).
"""
import os
import math
import heapq
from collections import deque
import matplotlib
matplotlib.use("Agg")  # guarda imágenes en archivo (sin abrir ventanas)
import matplotlib. pyplot as plt
import networkx as nx
#
================================================================
============
# PARTE 1: MARCO GENERICO DE BUSQUEDA
# ----------------------------------------------------------------------
# Un problema de búsqueda se define con 5 componentes (Russell y Norvig, 2020):
# 1) Estado inicial 2) Acciones(s) 3) Resultado (s, a)
# 4) Costo de paso c(s,a,s') 5) Prueba de meta
# Si el problema se define asi, el ESPACIO DE ESTADOS es un grafo dirigido y
# ponderado: nodo = estado, arista dirigida = accion, peso = costo del paso.
```

```
#
================================================================
============
class ProblemaBusqueda:
"""Interfaz abstracta: cada problema concreto hereda de aquí y define sus
5 componentes. Los algoritmos de abajo solo usan estos metodos, por eso
sirven para cualquier problema."""

estado_inicial = None    # (1) estado en el que empieza la búsqueda
def acciones (self, s):   # (2) lista de acciones aplicables en el estado s
raise NotImplementedError
def resultado (self, s, a):     # (3) modelo de transición: estado que resulta de
aplicar a en s
raise NotImplementedError
def costo (self, s, a, s2): # (4) costo de ir de s a s2 con la accion a (debe ser > 0)
return 1
def es_meta(self, s):  # (5). el estado s cumple el objetivo?
raise NotImplementedError
# ----------------------------------------------------------------------
# 2.2 ESPACIO DE BUSQUEDA: generador de estados
# ----------------------------------------------------------------------
def generar_espacio_estados(problema):
"""Recorre TODOS los estados alcanzables desde el inicial y construye el
grafo dirigido y ponderado (nx. DiGraph) del espacio de estados."""
G = nx.DiGraph() # grafo dirigido (las aristas tienen sentido)
inicio = problema.estado_inicial
G.add_node(inicio)    # el estado inicial es el primer nodo
visitados = {inicio}   # estados ya descubiertos (evita repetirlos)
cola = deque([inicio])    # cola FIFO: exploracion por niveles
while cola:
s = cola. popleft() # saca el siguiente estado por expandir
if problema.es_meta(s): # un estado meta ya no se expande
continue
for a in problema. acciones(s): # prueba cada accion aplicable
s2 = problema. resultado (s, a) # estado al que llega con esa accion
# arista s -> s2 con el nombre de la accion y su costo como peso
G.add_edge (s, s2, accion=a, weight=problema. costo(s, a, s2))
if s2 not in visitados: # si es un estado nuevo, hay que explorarlo
visitados.add(s2)
cola. append (s2)
return G
# ----------------------------------------------------------------------
# 2.1 ALGORITMOS DE BUSQUEDA CIEGA (búsqueda en grafo con lista de
explorados)
# Cada uno devuelve un diccionario con:
# camino (lista de estados), acciones, costo, expandidos, generados
# "expandidos" = estados cuyos sucesores se calcularon; "generados" = estados
```

```
# descubiertos. Ambos miden el esfuerzo (tiempo) del algoritmo.
# ---------------------------------------------------------------------------
def _reconstruir (padres, s):
"""Sigue los punteros 'padre' desde la meta hasta el inicio y devuelve el
camino y las acciones en orden de inicio -> meta."""
camino, acciones = [s], []

while padres[s][0] is not None: # el inicial tiene padre None
s, a = padres[s] # retrocede al padre y recuerda la accion
camino. append (s)
acciones. append (a)
camino. reverse () # estaban de meta a inicio: se invierten
acciones. reverse()
return camino, acciones
def _costo_camino (problema, camino, acciones):
"""Suma el costo de cada paso del camino."""
return sum (problema. costo(camino[i], acciones[i], camino[i + 1])
for i in range (len (acciones)))
def bfs (problema):
"""BUSQUEDA EN ANCHURA. Usa cola FIFO. Encuentra la solución con MENOS
PASOS; solo es óptima en costo si todos los pasos cuestan lo mismo."""
s0 = problema.estado_inicial
padres = {s0: (None, None)} # estado -> (padre, accion); tambien marca los
descubiertos
frontera = deque([s0]) # cola de estados por expandir
expandidos, generados = 0, 1
if problema.es_meta(s0):
return dict(camino=[s0], acciones=[], costo=0, expandidos=0, generados=1)
while frontera:
s = frontera.popleft()   # el más antiguo primero (por niveles)
expandidos += 1
for a in problema.acciones(s):
s2 = problema.resultado(s, a)
if s2 not in padres: # solo estados nuevos
padres[s2] = (s, a)
generados += 1
if problema.es_meta(s2): # BFS puede probar la meta al generar
c, ac = _reconstruir(padres, s2)
return dict(camino=c, acciones=ac, costo=_costo_camino(problema, c, ac),
expandidos=expandidos, generados=generados)
frontera.append(s2)
return None    # no hay solución
def dfs(problema):

"""BUSQUEDA EN PROFUNDIDAD. Usa pila LIFO. Con lista de explorados es
completa en espacios finitos, pero NO garantiza solución óptima."""
```

```
s0 = problema.estado_inicial
padres = {s0: (None, None)}
pila = [s0] # pila de estados por explorar
explorados = set() # estados ya sacados de la pila
expandidos, generados = 0, 1
while pila:
s = pila.pop() # el ultimo en entrar sale primero (profundiza)
if s in explorados:
continue
explorados.add(s)
if problema.es_meta(s):
c, ac = _reconstruir(padres, s)
return dict(camino=c, acciones=ac, costo=_costo_camino(problema, c, ac),
expandidos=expandidos, generados=generados)
expandidos += 1
for a in reversed(problema.acciones(s)):   # reversed: explora en el orden natural
s2 = problema.resultado(s, a)
if s2 not in explorados:
if s2 not in padres:
generados += 1
padres[s2] = (s, a)
pila.append(s2)
return None
def ucs(problema):

"""BUSQUEDA DE COSTO UNIFORME (Dijkstra). Usa cola de prioridad ordenada
por el costo acumulado g(n). Es COMPLETA y OPTIMA si todos los costos > 0."""

s0 = problema.estado_inicial
contador = 0 # desempate para que el heap nunca compare estados
frontera = [(0, contador, s0)]    # montículo de (costo acumulado, desempate,
estado)
padres = {s0: (None, None)}
g = {s0: 0}      # mejor costo conocido hasta cada estado
explorados = set()
expandidos, generados = 0, 1
while frontera:
costo, _, s = heapq.heappop(frontera)      # saca el estado de MENOR costo
acumulado
if s in explorados:    # entrada vieja (ya hay una mejor): se ignora
continue
if problema.es_meta(s):      # la meta se prueba al EXTRAER: así el costo es
mínimo
c, ac = _reconstruir(padres, s)
return dict(camino=c, acciones=ac, costo=costo,
expandidos=expandidos, generados=generados)
explorados.add(s)
```

```
expandidos += 1
for a in problema.acciones(s):
s2 = problema.resultado(s, a)
nuevo = costo + problema.costo(s, a, s2)          # costo de llegar a s2 pasando por
s
if s2 not in g or nuevo < g[s2]:   #. encontramos un camino mejor?
g[s2] = nuevo

padres[s2] = (s, a)
contador += 1
generados += 1
heapq.heappush(frontera, (nuevo, contador, s2))
return None
def factor_ramificacion(G):

"""Factor de ramificación promedio b = (no de aristas) / (no de nodos con
salida)."""

con_salida = [n for n in G if G.out_degree(n) > 0]
return G.number_of_edges() / max(1, len(con_salida))

# ----------------------------------------------------------------------------
# ANALISIS DE COMPLEJIDAD (teórico + medido). Lo imprime cada programa.
# Notación (Russell y Norvig, 2020):
# b = factor de ramificacion d = profundidad de la solución (no dé pasos)
# m = profundidad máxima del espacio C* = costo de la solución optima
# eps = costo mínimo de un paso |S|, |A| = no de estados y de transiciones
# ----------------------------------------------------------------------------

def analisis_complejidad(nombre, G, res, unidad):
n, m_aristas = G.number_of_nodes(), G.number_of_edges()
b = factor_ramificacion(G)
u = res["UCS"]
d, C = len(u["acciones"]), u["costo"]
eps = min(w for _, _, w in G.edges(data="weight"))
k = 1 + int(C // eps) # exponente de UCS: 1 + piso(C*/eps)
print(f"\n===== ANALISIS DE COMPLEJIDAD — {nombre} =====")
print(f"Datos medidos: |S|={n} |A|={m_aristas} b={b:.2f} d={d} "
f"C*={C} {unidad} eps={eps} {unidad}")
print("\nCotas teoricas SIN lista de explorados (arbol de busqueda):")
print(f" BFS: tiempo O(b^d) = {b:.2f}^{d} ≈ {b ** d:,.0f} nodos | espacio O(b^d)")
print(f" DFS: tiempo O(b^m) (m = prof. maxima) | espacio O(b · m) | no optimo")
print(f" UCS: tiempo O(b^(1+C*/eps)) = {b:.2f}^{k} ≈ {b ** k:,.0f} nodos | espacio
igual | optimo")
print("\nCotas CON lista de explorados (lo que implementa este programa):")
print(f" BFS y DFS: O(|S|+|A|) = O({n}+{m_aristas}) = {n + m_aristas} operaciones")
```

```
print(f" UCS con monticulo: O((|S|+|A|)log|S|) ≈ {(n + m_aristas) * math.log2(n):,.0f}
operaciones")
print(" Espacio: O(|S|) en los tres (guardan los estados explorados).")
print("\nResultados medidos (esfuerzo real de cada algoritmo):")
print(f"
{'Algoritmo':<10}{'Costo':>8}{'Pasos':>8}{'Expandidos':>12}{'Generados':>11}")
for nom in ("BFS", "DFS", "UCS"):
r = res[nom]
print(f"
{nom:<10}{r['costo']:>8}{len(r['acciones']):>8}{r['expandidos']:>12}{r['generados']:>1
1}")
print(f"\nConclusion: la cota b^d (≈{b ** d:,.0f}) seria inviable, pero la lista de
explorados "
f"limita el trabajo a los {n} estados del espacio; UCS expandio {u['expandidos']} de
{n} "
f"({100 * u['expandidos'] / n:.0f}%) y es el unico que garantiza el costo minimo.")

#
============================================================
============
# PARTE 2: PROBLEMA 1 — RUTA DE MENOR TIEMPO EN EL METRO DE LA
CDMX
#
============================================================
============
# ---- Datos del problema (supuestos del modelo) ----------------------------

T_TRAMO = 2      # minutos entre dos estaciones consecutivas (valor medio
aproximado)
T_TRANSBORDO = 5          # minutos por hacer un transbordo (caminar +
esperar, aproximado)
 # Estaciones de las líneas 1, 2 y 3 en orden de recorrido (datos de entrada)
LINEAS = {
1: ["Observatorio", "Tacubaya", "Juanacatlan", "Chapultepec", "Sevilla",
"Insurgentes",
"Cuauhtemoc", "Balderas", "Salto del Agua", "Isabel la Catolica", "Pino Suarez",
"Merced", "Candelaria", "San Lazaro", "Moctezuma", "Balbuena",
"Boulevard Puerto Aereo", "Gomez Farias", "Zaragoza", "Pantitlan"],
2: ["Cuatro Caminos", "Panteones", "Tacuba", "Cuitlahuac", "Popotla", "Colegio
Militar",
"Normal", "San Cosme", "Revolucion", "Hidalgo", "Bellas Artes", "Allende",
"Zócalo", "Pino Suarez", "San Antonio Abad", "Chabacano", "Viaducto", "Xola",
"Villa de Cortes", "Nativitas", "Portales", "Ermita", "General Anaya", "Tasquena"],
3: ["Indios Verdes", "Deportivo 18 de Marzo", "Potrero", "La Raza", "Tlatelolco",
"Guerrero", "Hidalgo", "Juarez", "Balderas", "Niños Heroes", "Hospital General",
"Centro Medico", "Etiopia", "Eugenia", "Division del Norte", "Zapata", "Coyoacan",
"Viveros", "Miguel Angel de Quevedo", "Copilco", "Universidad"],
```

```
}
ORIGEN = ("Universidad", 3) # estado inicial: en Universidad, viajando en la Linea
3
DESTINO = "Zócalo"        # meta: llegar a Zócalo (en cualquier linea)
class RutaMetro(ProblemaBusqueda):

"""FORMULACION DEL PROBLEMA (los 5 componentes).
Estado: (estacion, linea en la que va el usuario)
Acciones: IR_A_<estacion vecina> en la misma linea (2 min)
TRANSBORDO_L<n> si la estacion pertenece también a otra linea (5 min)
Meta: estar en la estacion Zócalo."""
def __init__(self):
self.estado_inicial = ORIGEN
# índice auxiliar: estacion -> {linea: posición dentro de esa linea}
self.idx = {}
for l, est in LINEAS.items():
for i, e in enumerate(est):
self.idx.setdefault(e, {})[l] = i
def acciones(self, s):
est, l = s
pos = self.idx[est][l]     # posición de la estacion en su línea



lista = LINEAS[l]
acc = []
if pos > 0:              # hay estacion anterior
acc.append(f"IR_A_{lista[pos - 1]}")
if pos < len(lista) - 1:               # hay estacion siguiente
acc.append(f"IR_A_{lista[pos + 1]}")
for otra in self.idx[est]:         # líneas que pasan por esta estacion
if otra != l:
acc.append(f"TRANSBORDO_L{otra}")
return acc
def resultado(self, s, a):
est, l = s
if a.startswith("TRANSBORDO_L"): # mismo lugar, otra linea
return (est, int(a[len("TRANSBORDO_L"):]))
return (a[len("IR_A_"):], l) # otra estacion, misma linea
def costo(self, s, a, s2):
# transbordar cuesta 5 min; avanzar un tramo cuesta 2 min (siempre > 0)
return T_TRANSBORDO if a.startswith("TRANSBORDO") else T_TRAMO
def es_meta(self, s):
return s[0] == DESTINO # no importa en que linea llegue
def resumen_ruta(res):
"""Agrupa el camino en tramos por linea (se corta en cada transbordo)."""
tramos, ini = [], res["camino"][0]
```

```
for i, a in enumerate(res["acciones"]):
if a.startswith("TRANSBORDO"):
tramos.append((ini, res["camino"][i])) # termina el tramo antes del transbordo
ini = res["camino"][i + 1] # el siguiente tramo empieza ya en la otra linea
tramos.append((ini, res["camino"][-1]))
return tramos
def dibujar(G, res, archivo):

"""Dibuja el espacio de estados (grafo dirigido ponderado) y resalta en rojo
la ruta optima. Cada linea del Metro se dibuja en un renglón."""

pos = {}                # posición (x, y) de cada nodo en el dibujo
for l, est in LINEAS.items():
for i, e in enumerate(est):
pos[(e, l)] = (i, -3 * l)
colores = {1: "#e8742a", 2: "#1f5fbf", 3: "#7aa13c"}
plt.figure(figsize=(15, 6.2))
ruta = set(res["camino"])
aristas_ruta = set(zip(res["camino"][:-1], res["camino"][1:]))
for l in LINEAS: # nodos de cada linea con su color
nodos = [n for n in G if n[1] == l]
nx.draw_networkx_nodes(G, pos, nodelist=nodos, node_size=70,
node_color=colores[l], alpha=0.85)
ar_normales = [e for e in G.edges if e not in aristas_ruta]
nx.draw_networkx_edges(G, pos, edgelist=ar_normales, edge_color="#bbbbbb",
width=0.6,
arrows=True, arrowsize=6, connectionstyle="arc3,rad=0.12")
nx.draw_networkx_edges(G, pos, edgelist=list(aristas_ruta),
edge_color="#d6001c",
width=2.6, arrows=True, arrowsize=12,
connectionstyle="arc3,rad=0.12")
nx.draw_networkx_nodes(G, pos, nodelist=list(ruta), node_size=110,
node_color="#d6001c", edgecolors="black")
for n in ruta: # nombres de la ruta, alternados arriba/abajo
x, y = pos[n]
arriba = (x % 2 == 0)
plt.text(x, y + (0.5 if arriba else -0.5), n[0], rotation=40, fontsize=6.5,
ha="left" if arriba else "right", va="bottom" if arriba else "top",
rotation_mode="anchor")
plt.ylim(-12.5, -1.5)
for l in LINEAS:
plt.text(-1.3, -3 * l, f"Linea {l}", color=colores[l], fontsize=11, fontweight="bold",
va="center")
plt.title("Problema 1 — Espacio de estados (estacion, linea): grafo dirigido y
ponderado\n"
"En rojo: ruta de costo uniforme (optima). Cada tramo = 2 min; transbordo = 5
min",
```

```
fontsize=11)
plt.axis("off")
plt.tight_layout()
plt.savefig(archivo, dpi=170) # guarda la imagen (no abre ventana)
plt.close()
def main():
# carpeta "figuras" junto a este archivo (se crea sola si no existe)
FIG = os.path.join(os.path.dirname(os.path.abspath(__file__)), "figuras")
os.makedirs(FIG, exist_ok=True)
p = RutaMetro() # 1) se crea el problema
G = generar_espacio_estados(p) # 2) se genera el espacio de estados (grafo)
print(f"[P1] |S| = {G.number_of_nodes()} estados, |A| = {G.number_of_edges()}
transiciones, "
f"b promedio = {factor_ramificacion(G):.2f}, "
f"dirigido = {G.is_directed()}, ponderado = {nx.is_weighted(G)}")
resultados = {"BFS": bfs(p), "DFS": dfs(p), "UCS": ucs(p)} # 3) se resuelve con 3
algoritmos
for nom, r in resultados.items():
print(f"[P1] {nom}: costo = {r['costo']} min, pasos = {len(r['acciones'])}, "
f"expandidos = {r['expandidos']}, generados = {r['generados']}")



print(" tramos:", resumen_ruta(r))
# 4) verificacion independiente: Dijkstra de NetworkX debe dar el mismo costo
optimo
metas = [n for n in G if p.es_meta(n)]
mejor = min(nx.dijkstra_path_length(G, p.estado_inicial, m) for m in metas)
print(f"[P1] Verificacion nx.dijkstra: costo optimo = {mejor} min "
f"(UCS = {resultados['UCS']['costo']} min) ->",
"OK" if mejor == resultados["UCS"]["costo"] else "ERROR")
assert mejor == resultados["UCS"]["costo"]
dibujar(G, resultados["UCS"], os.path.join(FIG, "p1_espacio_estados.png")) # 5)
diagrama del grafo
analisis_complejidad("Problema 1 (Metro CDMX)", G, resultados, "min") # 6)
complejidad
return p, G, resultados
if __name__ == "__main__":
main()
```

Código del Problema 2 comentado

A.2 problema2/problema2.py (Problema 2: marco de búsqueda + robot de almacén + complejidad)

```
"""
problema2.py -- Actividad 2 (2.1 Problemas de busqueda + 2.2 Espacios de
busqueda)
PROBLEMA 2: robot de almacen que recoge un paquete en P y lo entrega en D.
Archivo unico: no necesita otros archivos. Requiere: pip install networkx matplotlib
Ejecutar: python problema2.py
Salida : resultados y analisis de complejidad en la terminal +
figuras/p2_mapa_ruta.png y figuras/p2_espacio_estados.png
"""
import os
import math
import heapq
from collections import deque
import matplotlib
matplotlib.use("Agg") # guarda imagenes en archivo (sin abrir ventanas)
import matplotlib.pyplot as plt
import networkx as nx
#
================================================================
============
# PARTE 1: MARCO GENERICO DE BUSQUEDA
# ----------------------------------------------------------------------
# Un problema de busqueda se define con 5 componentes (Russell y Norvig,
2020):
# 1) Estado inicial 2) Acciones(s) 3) Resultado(s, a)
# 4) Costo de paso c(s,a,s') 5) Prueba de meta
# Si el problema se define asi, el ESPACIO DE ESTADOS es un grafo dirigido y
# ponderado: nodo = estado, arista dirigida = accion, peso = costo del paso.
#
================================================================
============
class ProblemaBusqueda:
"""Interfaz abstracta: cada problema concreto hereda de aqui y define sus
5 componentes. Los algoritmos de abajo solo usan estos metodos, por eso
sirven para cualquier problema."""
estado_inicial = None # (1) estado en el que empieza la busqueda
def acciones(self, s): # (2) lista de acciones aplicables en el estado s
raise NotImplementedError
def resultado(self, s, a): # (3) modelo de transicion: estado que resulta de aplicar a
en s
raise NotImplementedError
def costo(self, s, a, s2): # (4) costo de ir de s a s2 con la accion a (debe ser > 0)
return 1
def es_meta(self, s): # (5) .el estado s cumple el objetivo?
raise NotImplementedError
```

```
# ----------------------------------------------------------------------
# 2.2 ESPACIO DE BUSQUEDA: generador de estados
# ----------------------------------------------------------------------
def generar_espacio_estados(problema):
"""Recorre TODOS los estados alcanzables desde el inicial y construye el
grafo dirigido y ponderado (nx.DiGraph) del espacio de estados."""
G = nx.DiGraph() # grafo dirigido (las aristas tienen sentido)
inicio = problema.estado_inicial
G.add_node(inicio) # el estado inicial es el primer nodo
visitados = {inicio} # estados ya descubiertos (evita repetirlos)
cola = deque([inicio]) # cola FIFO: exploracion por niveles
while cola:
s = cola.popleft() # saca el siguiente estado por expandir
if problema.es_meta(s): # un estado meta ya no se expande
continue
for a in problema.acciones(s): # prueba cada accion aplicable
s2 = problema.resultado(s, a) # estado al que llega con esa accion
# arista s -> s2 con el nombre de la accion y su costo como peso
G.add_edge(s, s2, accion=a, weight=problema.costo(s, a, s2))
if s2 not in visitados: # si es un estado nuevo, hay que explorarlo
visitados.add(s2)
cola.append(s2)

return G
# ----------------------------------------------------------------------
# 2.1 ALGORITMOS DE BUSQUEDA CIEGA (busqueda en grafo con lista de
explorados)
# Cada uno devuelve un diccionario con:
# camino (lista de estados), acciones, costo, expandidos, generados
# "expandidos" = estados cuyos sucesores se calcularon; "generados" = estados
# descubiertos. Ambos miden el esfuerzo (tiempo) del algoritmo.
# ----------------------------------------------------------------------
def _reconstruir(padres, s):
"""Sigue los punteros 'padre' desde la meta hasta el inicio y devuelve el
camino y las acciones en orden de inicio -> meta."""
camino, acciones = [s], []
while padres[s][0] is not None: # el inicial tiene padre None
s, a = padres[s] # retrocede al padre y recuerda la accion
camino.append(s)
acciones.append(a)
camino.reverse() # estaban de meta a inicio: se invierten
acciones.reverse()
return camino, acciones
def _costo_camino(problema, camino, acciones):
"""Suma el costo de cada paso del camino."""
return sum(problema.costo(camino[i], acciones[i], camino[i + 1])
for i in range(len(acciones)))
```

```
def bfs(problema):
"""BUSQUEDA EN ANCHURA. Usa cola FIFO. Encuentra la solucion con MENOS
PASOS; solo es optima en costo si todos los pasos cuestan lo mismo."""
s0 = problema.estado_inicial
padres = {s0: (None, None)} # estado -> (padre, accion); tambien marca los
descubiertos
frontera = deque([s0]) # cola de estados por expandir
expandidos, generados = 0, 1
if problema.es_meta(s0):
return dict(camino=[s0], acciones=[], costo=0, expandidos=0, generados=1)
while frontera:
s = frontera.popleft() # el mas antiguo primero (por niveles)
expandidos += 1
for a in problema.acciones(s):
s2 = problema.resultado(s, a)
if s2 not in padres: # solo estados nuevos
padres[s2] = (s, a)
generados += 1
if problema.es_meta(s2): # BFS puede probar la meta al generar
c, ac = _reconstruir(padres, s2)
return dict(camino=c, acciones=ac, costo=_costo_camino(problema, c, ac),
expandidos=expandidos, generados=generados)
frontera.append(s2)
return None      # no hay solución
def dfs(problema):

"""BUSQUEDA EN PROFUNDIDAD. Usa pila LIFO. Con lista de explorados es
completa en espacios finitos, pero NO garantiza solución óptima."""

s0 = problema.estado_inicial
padres = {s0: (None, None)}
pila = [s0] # pila de estados por explorar
explorados = set() # estados ya sacados de la pila
expandidos, generados = 0, 1
while pila:
s = pila.pop()              # el último en entrar sale primero (profundiza)
if s in explorados:
continue
explorados.add(s)
if problema.es_meta(s):
c, ac = _reconstruir(padres, s)
return dict(camino=c, acciones=ac, costo=_costo_camino(problema, c, ac),
expandidos=expandidos, generados=generados)
expandidos += 1
for a in reversed(problema.acciones(s)): # reversed: explora en el orden natural
s2 = problema.resultado(s, a)
if s2 not in explorados:
```

```
if s2 not in padres:
generados += 1
padres[s2] = (s, a)
pila.append(s2)
return None
def ucs(problema):
"""BUSQUEDA DE COSTO UNIFORME (Dijkstra). Usa cola de prioridad ordenada
por el costo acumulado g(n). Es COMPLETA y OPTIMA si todos los costos > 0."""
s0 = problema.estado_inicial
contador = 0 # desempate para que el heap nunca compare estados
frontera = [(0, contador, s0)] # monticulo de (costo acumulado, desempate, estado)
padres = {s0: (None, None)}
g = {s0: 0} # mejor costo conocido hasta cada estado
explorados = set()
expandidos, generados = 0, 1
while frontera:



costo, _, s = heapq.heappop(frontera) # saca el estado de MENOR costo
acumulado
if s in explorados: # entrada vieja (ya hay una mejor): se ignora
continue
if problema.es_meta(s): # la meta se prueba al EXTRAER: asi el costo es minimo
c, ac = _reconstruir(padres, s)
return dict(camino=c, acciones=ac, costo=costo,
expandidos=expandidos, generados=generados)
explorados.add(s)
expandidos += 1
for a in problema.acciones(s):
s2 = problema.resultado(s, a)
nuevo = costo + problema.costo(s, a, s2) # costo de llegar a s2 pasando por s
if s2 not in g or nuevo < g[s2]: # .encontramos un camino mejor?
g[s2] = nuevo
padres[s2] = (s, a)
contador += 1
generados += 1
heapq.heappush(frontera, (nuevo, contador, s2))
return None
def factor_ramificacion(G):
"""Factor de ramificacion promedio b = (no de aristas) / (no de nodos con
salida)."""
con_salida = [n for n in G if G.out_degree(n) > 0]
return G.number_of_edges() / max(1, len(con_salida))
# ----------------------------------------------------------------------
# ANALISIS DE COMPLEJIDAD (teorico + medido). Lo imprime cada programa.
# Notacion (Russell y Norvig, 2020):
```

```
# b = factor de ramificacion d = profundidad de la solucion (no de pasos)
# m = profundidad maxima del espacio C* = costo de la solucion optima
# eps = costo minimo de un paso |S|, |A| = no de estados y de transiciones
# ----------------------------------------------------------------------
def analisis_complejidad(nombre, G, res, unidad):
n, m_aristas = G.number_of_nodes(), G.number_of_edges()
b = factor_ramificacion(G)
u = res["UCS"]
d, C = len(u["acciones"]), u["costo"]
eps = min(w for _, _, w in G.edges(data="weight"))
k = 1 + int(C // eps) # exponente de UCS: 1 + piso(C*/eps)
print(f"\n===== ANALISIS DE COMPLEJIDAD — {nombre} =====")
print(f"Datos medidos: |S|={n} |A|={m_aristas} b={b:.2f} d={d} "
f"C*={C} {unidad} eps={eps} {unidad}")
print("\nCotas teoricas SIN lista de explorados (arbol de busqueda):")
print(f" BFS: tiempo O(b^d) = {b:.2f}^{d} ≈ {b ** d:,.0f} nodos | espacio O(b^d)")
print(f" DFS: tiempo O(b^m) (m = prof. maxima) | espacio O(b · m) | no optimo")
print(f" UCS: tiempo O(b^(1+C*/eps)) = {b:.2f}^{k} ≈ {b ** k:,.0f} nodos | espacio
igual | optimo")
print("\nCotas CON lista de explorados (lo que implementa este programa):")
print(f" BFS y DFS: O(|S|+|A|) = O({n}+{m_aristas}) = {n + m_aristas} operaciones")
print(f" UCS con monticulo: O((|S|+|A|)log|S|) ≈ {(n + m_aristas) * math.log2(n):,.0f}
operaciones")
print(" Espacio: O(|S|) en los tres (guardan los estados explorados).")
print("\nResultados medidos (esfuerzo real de cada algoritmo):")
print(f"
{'Algoritmo':<10}{'Costo':>8}{'Pasos':>8}{'Expandidos':>12}{'Generados':>11}")
for nom in ("BFS", "DFS", "UCS"):
r = res[nom]
print(f"
{nom:<10}{r['costo']:>8}{len(r['acciones']):>8}{r['expandidos']:>12}{r['generados']:>1
1}")
print(f"\nConclusion: la cota b^d (≈{b ** d:,.0f}) seria inviable, pero la lista de
explorados "
f"limita el trabajo a los {n} estados del espacio; UCS expandio {u['expandidos']} de
{n} "
f"({100 * u['expandidos'] / n:.0f}%) y es el único que garantiza el costo minimo.")
#
==========================================================================
============
# PARTE 2: PROBLEMA 2 — ROBOT DE ALMACEN (recoger un paquete y
entregarlo)
#
==========================================================================
============
# ---- Datos del problema ------------------------------------------------
```

```
# Plano del almacén (5 filas x 7 columnas):
# R = base del robot (estado inicial) P = anaquel donde recoge el paquete
# D = muelle de entrega # = estante (obstáculo)
# ~ = pasillo congestionado (avanzar cuesta más) . = pasillo libre
MAPA = [
"R...#..",
".#..#.D",
".#.~~..",
"...#.#.",
".P.....",
]
C_NORMAL, C_CONGEST, C_ACCION = 1, 3, 2 # costos en segundos: mover / mover a zona ~ / recoger o entregar
class RobotAlmacen(ProblemaBusqueda):
"""FORMULACION DEL PROBLEMA (los 5 componentes).
Estado: (fila, columna, fase) fase 0 = sin paquete, 1 = con paquete, 2 = entregado
Acciones: N, S, E, O (mover una celda), RECOGER (solo en P, fase 0), ENTREGAR (solo en D, fase 1)
Meta: fase == 2.
La fase hace el grafo DIRIGIDO: RECOGER y ENTREGAR no se pueden deshacer."""
MOV = {"N": (-1, 0), "S": (1, 0), "E": (0, 1), "O": (0, -1)} # desplazamiento (fila, col)
def __init__(self, mapa=MAPA):
self.mapa = mapa


self.nf, self.nc = len(mapa), len(mapa[0])

for r, fila in enumerate(mapa): # localiza R, P y D en el plano

for c, ch in enumerate(fila):

if ch == "R":

self.estado_inicial = (r, c, 0) # empieza en R, sin paquete

elif ch == "P":

self.P = (r, c)

elif ch == "D":

self.D = (r, c)

def _libre(self, r, c):

""".La celda existe dentro del plano y no es un estante?"""

return 0 <= r < self.nf and 0 <= c < self.nc and self.mapa[r][c] != "#"
```

```
def acciones(self, s):
r, c, f = s
if f == 2: # ya entrego: no hay mas acciones
return []
acc = []
for a, (dr, dc) in self.MOV.items(): # movimientos posibles
if self._libre(r + dr, c + dc):
acc.append(a)
if f == 0 and (r, c) == self.P: # solo puede recoger estando en P sin paquete
acc.append("RECOGER")
if f == 1 and (r, c) == self.D: # solo puede entregar en D con paquete
acc.append("ENTREGAR")
return acc
def resultado(self, s, a):
r, c, f = s
if a == "RECOGER":
return (r, c, 1) # misma celda, ahora con paquete
if a == "ENTREGAR":
return (r, c, 2) # misma celda, paquete entregado (meta)
dr, dc = self.MOV[a]
return (r + dr, c + dc, f) # se mueve; la fase no cambia
def costo(self, s, a, s2):
if a in ("RECOGER", "ENTREGAR"):
return C_ACCION # 2 s
# moverse a una celda congestionada cuesta 3 s; a una libre, 1 s
return C_CONGEST if self.mapa[s2[0]][s2[1]] == "~" else C_NORMAL
def es_meta(self, s):
```

```
return s[2] == 2 # fase 2 = paquete entregado
def dibujar_mapa(p, res, archivo):
"""Dibuja el plano del almacen y la ruta optima encima."""
fig, ax = plt.subplots(figsize=(7.5, 5.4))
col = {"#": "#444444", "~": "#f4b183", ".": "#ffffff", "R": "#c6e0b4", "P": "#9dc3e6",
"D": "#ffe699"}
for r in range(p.nf):
for c in range(p.nc):
ch = p.mapa[r][c]
ax.add_patch(plt.Rectangle((c, p.nf - r - 1), 1, 1, fc=col[ch], ec="#888888"))
if ch in "RPD": # rotula R, P y D
ax.text(c + .5, p.nf - r - .5, ch, ha="center", va="center", fontsize=15,
fontweight="bold")
if ch == "~": # marca el costo triple
ax.text(c + .5, p.nf - r - .5, "×3", ha="center", va="center", fontsize=10,
color="#843c0c")
xs = [s[1] + .5 for s in res["camino"]] # centro de cada celda de la ruta
ys = [p.nf - s[0] - .5 for s in res["camino"]]
ax.plot(xs, ys, "-o", color="#d6001c", lw=2.4, ms=4)
ax.set_xlim(0, p.nc); ax.set_ylim(0, p.nf); ax.set_aspect("equal"); ax.axis("off")
ax.set_title(f"Problema 2 — Plano del almacen y ruta optima (costo = {res['costo']}
s)",
fontsize=11)
plt.tight_layout(); plt.savefig(archivo, dpi=170); plt.close()
def dibujar_grafo(p, G, res, archivo):
"""Dibuja el espacio de estados en tres capas (fase 0, fase 1 y meta):
grafo dirigido y ponderado, con la ruta optima en rojo."""
```

```
desplaz = {0: 0, 1: p.nc + 2, 2: 2 * (p.nc + 2)}          # desplazamiento horizontal de cada capa

pos = {s: (s[1] + desplaz[s[2]], -s[0]) for s in G}       # posicion (x, y) de cada estado

plt.figure(figsize=(15, 4.8))

ruta = res["camino"]

ar_ruta = set(zip(ruta[:-1], ruta[1:]))

no_ruta = [e for e in G.edges if e not in ar_ruta]

tipo = {0: "#9dc3e6", 1: "#f4b183", 2: "#a9d18e"} # color por fase

nx.draw_networkx_nodes(G, pos, node_size=55, node_color=[tipo[s[2]] for s in G])

nx.draw_networkx_edges(G, pos, edgelist=no_ruta, edge_color="#c8c8c8",
width=0.5,

arrows=True, arrowsize=5, connectionstyle="arc3,rad=0.1")

nx.draw_networkx_edges(G, pos, edgelist=list(ar_ruta), edge_color="#d6001c",
width=2.4,

arrows=True, arrowsize=10, connectionstyle="arc3,rad=0.1")

nx.draw_networkx_nodes(G, pos, nodelist=ruta, node_size=70,
node_color="#d6001c")

for f, txt in ((0, "Capa 0: sin paquete"), (1, "Capa 1: con paquete"), (2, "Meta")):

plt.text(desplaz[f] + (p.nc - 1) / 2, 0.7, txt, ha="center", fontsize=10,

fontweight="bold")

plt.ylim(-4.6, 1.3)

plt.title("Problema 2 — Espacio de estados (fila, col, fase): grafo dirigido y
ponderado.\n"

"Las aristas RECOGER y ENTREGAR son de un solo sentido (capa 0→1→meta)",



fontsize=11, pad=18)
plt.axis("off"); plt.tight_layout(); plt.savefig(archivo, dpi=170); plt.close()
def main():
# carpeta "figuras" junto a este archivo (se crea sola si no existe)
FIG = os.path.join(os.path.dirname(os.path.abspath(__file__)), "figuras")
os.makedirs(FIG, exist_ok=True)
```

```
p = RobotAlmacen() # 1) se crea el problema
G = generar_espacio_estados(p) # 2) se genera el espacio de estados (grafo)
print(f"[P2] |S| = {G.number_of_nodes()} estados, |A| = {G.number_of_edges()}
transiciones, "
f"b promedio = {factor_ramificacion(G):.2f}, "
f"dirigido = {G.is_directed()}, ponderado = {nx.is_weighted(G)}")
res = {"BFS": bfs(p), "DFS": dfs(p), "UCS": ucs(p)} # 3) se resuelve con 3
algoritmos
for nom, r in res.items():
print(f"[P2] {nom}: costo = {r['costo']} s, pasos = {len(r['acciones'])}, "
f"expandidos = {r['expandidos']}, generados = {r['generados']}")
print("[P2] Acciones UCS:", " ".join(res["UCS"]["acciones"]))
# 4) verificacion independiente con Dijkstra de NetworkX
metas = [n for n in G if p.es_meta(n)]
mejor = min(nx.dijkstra_path_length(G, p.estado_inicial, m) for m in metas)
print(f"[P2] Verificacion nx.dijkstra: costo optimo = {mejor} s "
f"(UCS = {res['UCS']['costo']} s) ->", "OK" if mejor == res["UCS"]["costo"] else
"ERROR")
assert mejor == res["UCS"]["costo"]
dibujar_mapa(p, res["UCS"], os.path.join(FIG, "p2_mapa_ruta.png")) # 5) plano
con la ruta
dibujar_grafo(p, G, res["UCS"], os.path.join(FIG, "p2_espacio_estados.png")) # 5)
grafo del espacio
analisis_complejidad("Problema 2 (Robot de almacen)", G, res, "s") # 6)
complejidad
return p, G, res
if __name__ == "__main__":
main()
```